Artifact Narrative – Enhanced CRUD Module & Dashboard

Artifact Description

The artifact for this enhancement is the Grazioso Salvare Rescue Dashboard and its supporting CRUD module originally developed in CS-340. The original version included a basic MongoDB CRUD class, simple rescue filters, and a Dash dashboard with limited error handling and no architectural separation. This enhancement focused on transforming the project into a professionally engineered, maintainable, and secure application.

Justification for Inclusion

I selected this artifact because it demonstrates full-stack engineering across backend, database, and UI layers. Enhancing it allowed me to showcase core software engineering skills such as modular architecture, secure coding, structured logging, exception handling, and maintainability improvements. These enhancements directly align with the Software Design & Engineering outcome and demonstrate my ability to produce production-quality code.

Key Enhancements Completed

1. Custom Exception Hierarchy I replaced generic exceptions and print statements with a structured exception hierarchy (AnimalShelterError, AnimalShelterValidationError, etc.).

Before: Errors printed to console with no categorization.

After: Errors are logged with severity levels and raised as typed exceptions, improving debugging and reliability.

2. Environment-Based Configuration Hardcoded credentials were removed and replaced with environment variables and a dedicated login_page.py module. This significantly improved security and maintainability.

3. Structured Logging All CRUD operations now use Python's logging module.

Before: print("Error inserting document")

After: logger.exception("Error inserting document") This provides timestamps, severity levels, and tracebacks.

4. Modular Architecture The dashboard was refactored to separate UI logic, business logic, and database access. Callback functions were cleaned, documented, and wrapped in error handling.

5. Improved Validation & Health Checks New validation helpers prevent unsafe operators, schema drift, and malformed updates. A new health() method provides connection diagnostics.

Testing Evidence

Testing confirmed that the enhancements worked as intended:

- Invalid operators such as {"$where": "..."} now correctly raise AnimalShelterValidationError.
- Missing required fields (e.g., "name") produce clear validation errors.
- Connection failures raise AnimalShelterConnectionError instead of crashing the dashboard.
- Logging output shows categorized errors with timestamps, improving traceability.
- Rescue filters now raise dedicated exceptions when malformed data is encountered.

These results demonstrate measurable improvements in reliability, maintainability, and security.

Reflection on the Enhancement Process

Enhancing this artifact taught me how to refactor legacy code into a clean, maintainable architecture. I learned how to apply structured logging, design custom exception hierarchies, and enforce validation rules that prevent unsafe database interactions. One challenge was ensuring that new validation logic did not break existing dashboard callbacks. I resolved these challenges through iterative testing and careful error handling.